Laburin

Memoria Técnica

Plataforma Web de Búsqueda y Seguimiento Laboral Automatizada con Motor de Scoring

Práctica Profesional Supervisada

Tecnicatura Universitaria en Tecnologías Web — Universidad Nacional del Oeste

Autor: Agustín Elisey — Docente: Viviana Esterkin

Segundo Cuatrimestre 2026

Índice

1. Resumen del proyecto

Problema
Objetivo
Alcance — qué entra
Alcance — qué no entra

2. Arquitectura

Stack
Por qué este stack
Estructura de carpetas

3. Modelo de datos

Notas del modelo

4. El motor de scoring

Cómo se calcula
Por qué configurable
Recálculo, no caché

5. Decisiones de diseño relevantes

5.1 Selector en mobile en vez de @dnd-kit
5.2 Importación manual en vez de Edge Function + cron
5.3 Dos fuentes externas en vez de las siete originales
5.4 Estado de borrador separado para los campos de peso
5.5 Umbral de inactividad: una columna por usuario, con efecto solo hacia adelante
5.6 Paginación en el servidor en vez de traer todo y filtrar en memoria
5.7 Roles en una tabla aparte, no como columna del perfil
5.8 El administrador lee una vista, no la tabla de perfiles
5.9 La bienvenida la manda un trigger de base de datos
5.10 El destinatario puede borrar sus mensajes
5.11 Claridad para quien no conoce la aplicación
5.12 Cabeceras de seguridad y política de contenido
5.13 Plantillas de mensaje con campos por completar

6. Seguridad y privacidad

Modelo de acceso
Qué datos ve el administrador
Aviso al usuario
Pruebas de seguridad
Revisión de seguridad de la aplicación
Límites

7. Testing

Metodología
Pase de QA

Pase E2E
Pase con Playwright
Pase de claridad y roles
Bugs de producción

8. Trabajo fuera del alcance original

9. Limitaciones conocidas y trabajo futuro

10. Conclusiones

Requerimientos funcionales
Requerimientos no funcionales
Ampliación acordada con la docente (08/10/2026)

1. Resumen del proyecto

Problema

Laburin parte de un antecedente funcional real: un script de Google Apps Script (jobbot-v5.gs) que el autor usaba para su propia búsqueda laboral. Ese script buscaba ofertas en 7 fuentes, las puntuaba con un motor de scoring hardcodeado y mandaba un mail diario con un link a una planilla de Google Sheets. Sus limitaciones centrales — identificadas en la etapa de relevamiento (SRS, §3.6) — eran: reglas de puntuación hardcodeadas e indeployables sin tocar código, ausencia de modelo relacional (cada corrida sobreescribía la planilla del día), y una salida puramente pasiva, sin seguimiento del ciclo de vida de una postulación después de que la oferta aparecía en el mail.

Objetivo

Construir una plataforma web que reemplace ese script por un sistema con persistencia relacional real, un motor de scoring configurable desde la interfaz (sin tocar código) y, sobre todo, una capa que no existía en el antecedente: el seguimiento activo de cada postulación a lo largo de su ciclo de vida, desde que se decide aplicar hasta que el proceso termina.

Alcance — qué entra

- Motor de scoring configurable por el usuario sobre 3 criterios: stack tecnológico, modalidad y ubicación.
- Carga de ofertas manual y por importación desde fuentes externas.
- Tablero Kanban de 6 estados para seguir cada postulación, con historial de cambios de estado.
- Ficha de Postulación con registro de interacciones (mails, llamados, entrevistas).
- Recordatorios automáticos por inactividad prolongada de una postulación.
- Panel de Métricas con indicadores operativos de toda la búsqueda.
- Autenticación multiusuario real: registro, inicio de sesión, recuperación de contraseña por email y configuración inicial guiada en 5 pasos.
- Ampliación acordada con la docente el 08/10/2026: un rol de administrador que ve el perfil de búsqueda de los candidatos y les envía mensajes, y una sección Mensajes para el candidato con una bienvenida automática (Secciones 5.7 a 5.10 y 6).
- Una interfaz pensada para quien no la conoce: secciones con nombres claros, ayuda en cada pantalla, estados vacíos que guían, tarjeta "Primeros pasos" y errores siempre en español (Sección 5.11).

Alcance — qué no entra

Reafirmado de la propuesta original (SRS, §8): bolsa de empleo pública para terceros o redes sociales profesionales, portales de publicación directa de ofertas o gestión de RRHH/contratos, y automatización de envío masivo/autónomo de currículums fuera de esta plataforma.

Además, el 08/10/2026 la docente (Viviana Esterkin) pidió que, junto al lado de quien busca trabajo, exista el otro lado: la persona que recibe y revisa lo que cargan quienes se registran, y que puede escribirles. Eso amplió el alcance original (una herramienta personal de seguimiento) de forma mínima y acotada, y se documenta como ampliación acordada, no como parte de la especificación original. La Sección 6 describe cómo se resguardan los datos.

Nombres de las pantallas: desde esa misma devolución, el Tablero se llama "Mis postulaciones", Analytics se llama "Mi progreso" y Configuración se llama "Mi perfil de búsqueda". Esta memoria usa los nombres anteriores cuando habla de decisiones tomadas antes del cambio.

Además, durante el desarrollo se redujeron deliberadamente algunos puntos que la especificación original sí contemplaba (automatización periódica completa de 7 fuentes, reglas fijas de scoring heredadas del script anterior). La Sección 10 compara, punto por punto, qué de la especificación original se cumplió y qué quedó afuera a propósito.

2. Arquitectura

Stack

Frontend: React 19 + TypeScript, sobre Vite 6 como bundler/dev server, con Tailwind CSS v4 para estilos (tokens de tema definidos con @theme, soporte de claro/oscuro vía prefers-color-scheme con override manual por data-theme). Ruteo con React Router v7. Gráficos del Panel de Métricas con Recharts. Testing unitario con Vitest.

Backend: Supabase (PostgreSQL gestionado + Auth + Row Level Security), sin servidor propio. Cada tabla de dominio tiene RLS activado por user_id, de forma que un usuario solo puede leer y escribir sus propias filas — esto se verificó también como parte del testing (ver Sección 7). Desde la ampliación del 08/10/2026 hay además un rol de administrador con permisos mínimos, acotados a nivel de columna (Sección 6).

Por qué este stack

- Supabase reemplaza la planilla de Google Sheets del antecedente por persistencia relacional real, con autenticación y control de acceso por fila ya resueltos, sin tener que levantar y mantener un backend propio — una decisión apropiada para el alcance y el tiempo disponible de una PPS.
- TypeScript con tipado estricto resultó necesario por la cantidad de entidades relacionadas entre sí (ofertas, postulaciones, criterios, interacciones, recordatorios) — los errores de forma de datos entre capas se detectan en compilación, no en producción.
- Tailwind v4 permitió iterar visualmente rápido sin mantener una hoja de estilos paralela, lo cual importó particularmente en la etapa de pulido de usabilidad/mobile, donde se iteró mucho sobre tamaños de touch target y breakpoints.

Estructura de carpetas

El proyecto organiza src/ por tipo de archivo, no por dominio/feature:

src/
components/ componentes UI reutilizables y de layout
layout/ AppLayout, Sidebar, Topbar
auth/ AuthLayout, CampoPassword, MedidorPassword
features/ flujos multi-paso con estado propio
onboarding/ OnboardingWizard + steps/ (Nombre y rol, Tecnologías, Modalidad, Nivel, Resumen)
pages/ una vista por ruta (Login, Recuperar y Nueva contraseña, Offers, Tablero, Settings, Analytics, Mensajes, Candidatos, Onboarding)
hooks/ useAuth, useRol, useOnboarding, useTheme, useMensajesNoLeidos
services/ un archivo por entidad, única capa que habla con Supabase
lib/ utilidades puras (errores, authErrores, validacionAuth, rol, candidatos, mensajes, ubicaciones, utils, passwordStrength, fuentes, seleccion, lotes, tags)
e2e/ suite de Playwright (fuera de src/, se corre con npm run test:e2e)
types/ interfaces TypeScript compartidas

La capa de services/ es la única que importa el cliente de Supabase: cada entidad del dominio (ofertas, postulaciones, interacciones, criteriosScoring, scoring, recordatorios, metricas, fuentesExternas, onboarding, roles, candidatos, mensajes) tiene su propio archivo, y las páginas consumen esas funciones en vez de armar queries propias. Esto fue lo que permitió, por ejemplo, verificar en el pase de testing E2E que un recálculo de scoring dentro de un service se propaga solo, sin que ningún componente de React tenga que "acordarse" de refrescar nada (ver Sección 4).

3. Modelo de datos

El esquema vive enteramente en PostgreSQL (Supabase), con RLS por user_id en cada tabla de dominio. El siguiente diagrama resume las entidades y sus relaciones reales, tal como quedaron definidas en las migraciones de supabase/migrations/. En verde están las tablas y la vista agregadas con la ampliación del 08/10/2026.

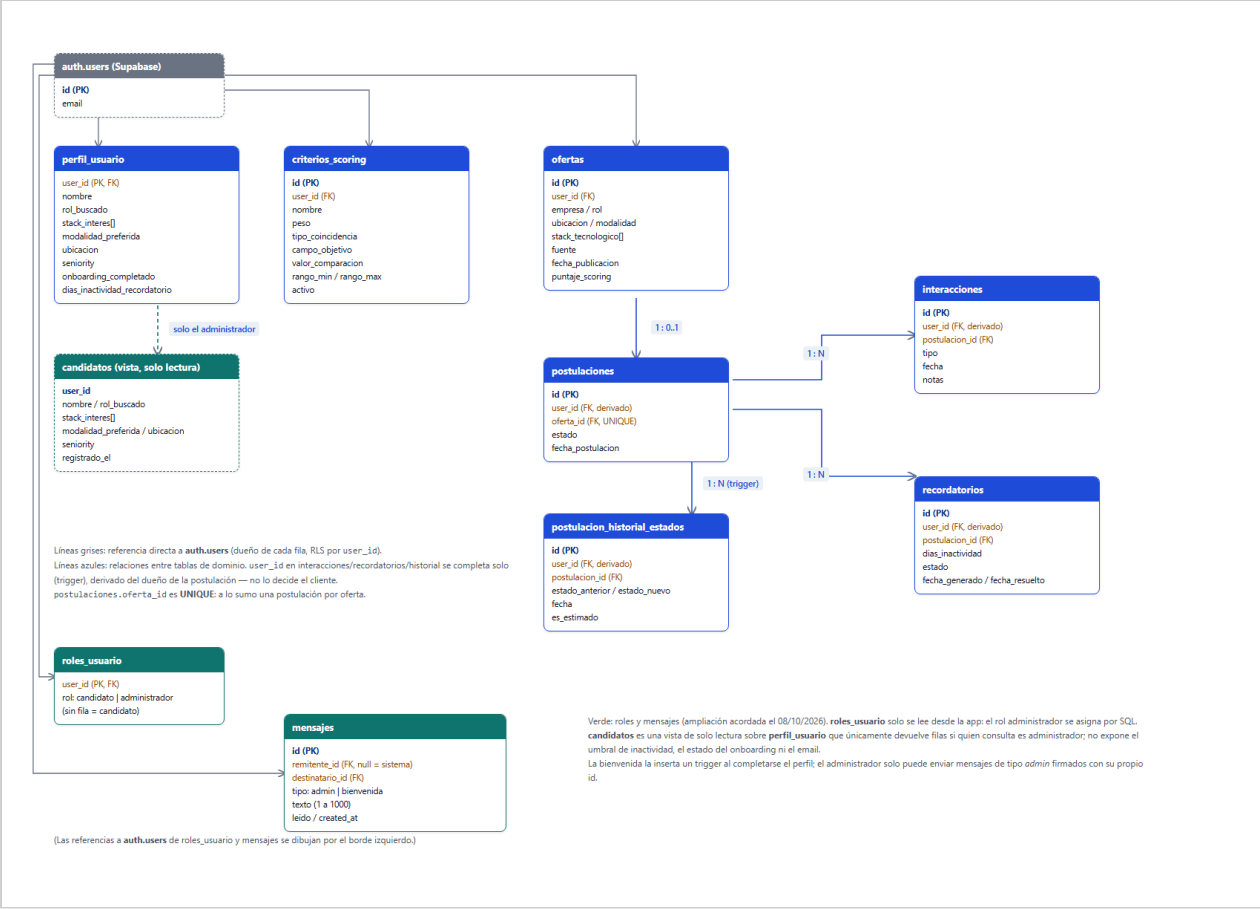


Diagrama entidad-relación del esquema real de Supabase.

Notas del modelo

- perfil_usuario.dias_inactividad_recordatorio guarda el umbral de inactividad de cada usuario (entero, default 7, con un CHECK que obliga a estar entre 1 y 90). El mismo rango y el mismo default 7 se validan en el frontend, de modo que la base es la última línea de defensa y no la única (Sección 5.5).
- Toda tabla de dominio (perfil_usuario, criterios_scoring, ofertas, postulaciones, interacciones, recordatorios, postulacion_historial_estados) referencia a auth.users mediante user_id, con RLS activado — este patrón se adoptó desde el modelado inicial (SRS, RNF-02), antes de que se decidiera construir la funcionalidad de multi-usuario completa (ver Sección 8).
- roles_usuario guarda el rol de cada persona (candidato o administrador); sin fila, es candidato. La aplicación solo puede leerla: no tiene permiso ni política de escritura, de modo que nadie puede escalar su propio rol, y el administrador se asigna por SQL (Sección 5.7).

- candidatos es una vista de solo lectura sobre perfil_usuario, con las columnas del perfil de búsqueda; solo devuelve filas si quien consulta es administrador (Sección 5.8).
- mensajes guarda remitente (nulo para los del sistema), destinatario, tipo (admin o bienvenida), texto de 1 a 1000 caracteres y si fue leído; la bienvenida la inserta un trigger (Sección 5.9).
- perfil_usuario.nombre es el nombre para mostrar, que ve el administrador. El email no se copia al perfil: el administrador no lo ve.
- postulaciones.oferta_id tiene una constraint UNIQUE: a lo sumo una Postulación por Oferta. La relación Oferta→Postulación es, en los hechos, 1 a 0..1.
- interacciones, recordatorios y postulacion_historial_estados derivan su user_id del dueño de la postulación mediante trigger — el cliente nunca lo decide directamente, lo cual cierra una vía de escritura cruzada entre usuarios incluso si el frontend tuviera un bug.
- postulacion_historial_estados se llena exclusivamente vía un trigger de base de datos (registrar_historial_estado, security definer, con RLS de solo lectura para el cliente) — cada cambio de estado queda registrado sin importar si vino del drag de desktop o del selector de mobile, porque ambos caminos terminan escribiendo la misma fila de postulaciones.
- recordatorios tiene un índice único parcial (recordatorios_un_activo_por_postulacion_idx) sobre postulacion_id donde estado='activo' — garantiza, a nivel de base, que nunca haya dos recordatorios activos simultáneos para la misma postulación.

4. El motor de scoring

Cómo se calcula

El cálculo del puntaje de una oferta es, deliberadamente, muy simple: suma el peso de cada criterio activo que la oferta matchea.

```
export function calcularScoring(
oferta: OfertaParaScoring,
criterios: CriterioScoring[]
): number {
return criterios
.filter((criterio) => criterio.activo)
.filter((criterio) => criterioMatchea(oferta, criterio))
.reduce((total, criterio) => total + criterio.peso, 0);
}
```

Cada criterio define un campo_objetivo de la oferta a mirar (stack_tecnologico, modalidad, ubicacion, etc.), un valor_comparacion, y un tipo_coincidencia: contiene (substring, case-insensitive), exacto (igualdad case-insensitive) o rango_numerico (entre rango_min y rango_max). criterioMatchea aplica la comparación que corresponda según el tipo.

Por qué configurable

Esto responde directamente a la limitación #1 identificada en el antecedente (SRS, §3.6): en el script v5.0 todo el peso de cada señal de scoring estaba hardcodeado en un objeto CONFIG, y cambiar una keyword o un peso requería editar y redeployar el script. Exponer peso, campo_objetivo y valor_comparacion como filas editables en criterios_scoring (tabla, no código) elimina esa fricción: el usuario ajusta sus prioridades desde Mi perfil de búsqueda (antes Configuración), sin pasar por un deploy.

Recálculo, no caché

Cada vez que se crea, edita o borra un criterio, o se crea/edita una oferta, el service correspondiente llama a recalcularTodosLosScores(userId) de forma síncrona antes de que la función async termine — no es un efecto que dependa de que un componente de React lo dispare por separado. El recálculo lee las ofertas por lotes (para no toparse con el límite de 1000 filas de una sola lectura) y escribe únicamente las filas cuyo puntaje cambió, de a diez en paralelo. Esto se verificó explícitamente contra una base real durante el pase de testing E2E: subir el peso de un criterio de 15 a 25 recalculó una oferta ya cargada de 30 a 40 puntos, sin que la página de Ofertas necesitara ningún tipo de invalidación de caché — porque no existe tal caché: cada página pide los datos de nuevo al montarse.

5. Decisiones de diseño relevantes

Esta sección cubre únicamente decisiones que tuvieron una alternativa real evaluada y descartada — no un recuento general de todo lo construido.

5.1 Selector en mobile en vez de @dnd-kit

El Tablero usa drag-and-drop nativo de HTML5 en desktop. La alternativa evaluada para que el Kanban también soportara arrastre táctil en mobile era sumar una librería dedicada (@dnd-kit u otra). Se descartó: el drag nativo de HTML5 es poco confiable sobre touch, y agregar una dependencia nueva solo para ese caso de uso no se justificaba frente a una alternativa más simple. En su lugar, cada tarjeta expone en mobile un selector "Mover a..." que llama exactamente a la misma función de cambio de estado que usa el drag de desktop — mismo camino de datos, dos interfaces de entrada. El costo aceptado es perder la sensación de arrastre en mobile; lo ganado es cero dependencias nuevas y un único punto de verdad para el cambio de estado, más simple de testear.

5.2 Importación manual en vez de Edge Function + cron

La especificación original (RNF-05) pedía que la búsqueda en fuentes externas corriera en un job programado, no bajo demanda del usuario. Se descartó esa automatización dentro del tiempo de esta PPS: el foco se puso en el motor de scoring configurable y el tablero de seguimiento — el valor nuevo central del proyecto — por sobre la automatización completa del pipeline de búsqueda. La importación quedó como un botón ("Importar ofertas remotas") que el usuario dispara cuando quiere. Aparte se agregó un GitHub Action con cron cada 3 días, pero cumple un propósito distinto: solo hace un ping liviano a Supabase para evitar que el proyecto gratuito se pause por inactividad, no reemplaza la búsqueda periódica que pedía RNF-05.

5.3 Dos fuentes externas en vez de las siete originales

El antecedente (v5.0) consultaba 7 fuentes, incluyendo el parseo de HTML del endpoint público de LinkedIn — identificado en la propia SRS (§3.2) como "la más frágil: no es JSON, se parsea con regex; cualquier cambio de markup la rompe". Se evaluó portar las 7 y se descartó: Remotive y Arbeitnow exponen API REST JSON pública y estable, mientras que sumar LinkedIn (parseo HTML no oficial) y las fuentes restantes (Indeed AR, GetOnBrd, Vacantes Digitales, Himalayas) hubiera sumado superficie de mantenimiento sin aportar directamente al objetivo central de la PPS. Se priorizó profundidad en scoring/tablero/ficha por sobre amplitud de fuentes.

5.4 Estado de borrador separado para los campos de peso

Durante el pase de QA se encontró que el campo "Peso" de un criterio existente se "pegaba" en 0 al editarlo: Number('') evalúa a 0, no a NaN, y como el input es controlado, React repintaba el campo con "0" antes de que el usuario terminara de borrarlo para escribir un valor nuevo. La alternativa evaluada fue un parche puntual (tratar value==='' como caso especial dentro del mismo estado numérico). Se descartó por no atacar la causa real: mezclar en un solo estado "lo que se está escribiendo" con "lo que está persistido" es estructuralmente proclive a este tipo de bug. La solución adoptada fue separar ambas cosas explícitamente — un estado de borradores en string (pesoDrafts),

independiente del valor numérico guardado, que solo se parsea y persiste al perder el foco del campo. El mismo patrón se reutilizó después para el campo "Umbral de Inactividad" (Sección 5.5).

5.5 Umbral de inactividad: una columna por usuario, con efecto solo hacia adelante

El umbral que dispara un recordatorio (RF-14) vivía hardcodeado en el motor y el campo de Configuración no hacía nada. Para persistirlo por usuario se evaluó crear una tabla de preferencias aparte y se descartó: es un único valor por cuenta, y perfil_usuario ya tiene RLS por user_id y el trigger de updated_at. Se agregó una columna dias_inactividad_recordatorio (entero, default 7, CHECK entre 1 y 90). El default de la base, el del motor y el que muestra la pantalla salen de la misma constante, para que no vuelva a pasar que la interfaz diga 30 y el motor use 7.

Qué pasa con los recordatorios ya generados cuando el usuario cambia el umbral: el cambio solo aplica hacia adelante. Los recordatorios activos no se tocan ni se recalculan hacia atrás, porque la persona puede haberlos visto y borrarlos sería sorprendente. Si baja el umbral, las postulaciones que ya superan el nuevo valor generan su recordatorio en la próxima carga; si lo sube, los activos quedan hasta que se resuelvan y no se generan otros nuevos hasta cumplir el nuevo plazo. La decisión de "está inactiva" es una función pura (estaInactiva, exige estrictamente más de N días), cubierta con tests unitarios en los bordes (exactamente N días, N-1, umbral 1 y umbral 90) y con un test E2E de la política.

5.6 Paginación en el servidor en vez de traer todo y filtrar en memoria

El listado de Ofertas traía todas las filas y filtraba en el navegador; como Supabase corta cada lectura en 1000 filas, pasado ese volumen se perdían ofertas sin ningún aviso. La alternativa más barata era seguir trayendo todo, por lotes, y filtrar en memoria; se descartó porque con miles de ofertas cargaría y mantendría todo en el cliente para mostrar treinta tarjetas. Se resolvió con paginación en el servidor (range() con conteo exacto) y los tres filtros (fuente, ubicación, puntaje mínimo) aplicados en la propia consulta.

El borrado masivo se rediseñó para que haga exactamente lo que dice. La casilla de la cabecera selecciona la página visible y lo aclara en su texto; cuando hay más coincidencias que las visibles aparece un enlace "Seleccionar las N ofertas que coinciden con los filtros", que trae solo los ids por lotes. "Eliminar seleccionadas (N)" borra siempre esos N. La deduplicación del import dejó de depender de tener las ofertas cargadas: consulta, por lotes y solo con las columnas necesarias, las claves ya guardadas de las fuentes que se importan. Las demás lecturas "de todo" (Analytics, Tablero, recordatorios, historial) leen por lotes con un helper común (leerEnLotes) con orden estable.

5.7 Roles en una tabla aparte, no como columna del perfil

La primera idea fue agregar una columna rol a perfil_usuario. Se descartó: esa tabla ya deja que cada usuario escriba su propia fila (el Onboarding lo necesita), así que con una columna de rol cualquiera podría hacerse administrador con un simple upsert, y habría que blindar insert, update y upsert con permisos por columna. Se creó roles_usuario, que los usuarios solo pueden leer: sin permiso de escritura ni política de escritura no existe ningún camino de escalada. Sin fila, la persona es candidata,

así que no hace falta un trigger al registrarse. El rol administrador se asigna a mano, por SQL, a las cuentas que indica el responsable del proyecto.

5.8 El administrador lee una vista, no la tabla de perfiles

La alternativa era darle al administrador una política de lectura sobre perfil_usuario. Se descartó porque habría expuesto la fila completa (por ejemplo, el umbral de recordatorios y el estado del onboarding). En su lugar, la vista candidatos expone solo nombre, rol buscado, tecnologías, modalidad, ubicación, nivel y fecha de registro, solo devuelve filas si es_administrador() es verdadera y excluye a los propios administradores.

Una vista simple es actualizable con los permisos de su dueño, y en Supabase los objetos nuevos nacen con permisos para los roles de la API. Por eso a la vista se le quitaron todos los permisos y se otorgó únicamente SELECT: por ella no se puede insertar, actualizar ni borrar. La misma regla (revoke all y recién después el permiso mínimo) se aplicó a cada tabla, vista y función nueva.

5.9 La bienvenida la manda un trigger de base de datos

Se evaluó insertar el mensaje desde la aplicación al terminar el Onboarding. Se descartó: un cliente podría saltearlo o falsificarlo, y la aplicación tendría que acordarse de hacerlo en cada camino. Un trigger sobre perfil_usuario lo inserta en el momento en que el perfil pasa a completo con un rol cargado, y un índice único parcial impide duplicarlo. Quien usa "Omitir por ahora" no lo recibe, porque "tu perfil quedó registrado" no sería cierto. El administrador, por su parte, solo puede enviar mensajes de tipo admin firmados con su propio id.

5.10 El destinatario puede borrar sus mensajes

Los mensajes pertenecen a quien los recibe: puede marcarlos como leídos (es lo único que puede cambiar de un mensaje; el texto, el remitente y el destinatario están protegidos por permisos de columna) y borrarlos. Es un control razonable de la bandeja propia, y además permite que las pruebas automáticas limpien sus datos sin una service role key. El costo aceptado: si la persona borra un mensaje, el administrador lo pierde de su historial de enviados.

5.11 Claridad para quien no conoce la aplicación

El 08/10/2026 la docente señaló que la aplicación funcionaba pero solo se entendía si ya se sabía usar. La respuesta fue un pase de claridad: secciones con nombres entendibles en el orden en que se usan, una línea de ayuda bajo cada título, estados vacíos con un botón hacia el paso siguiente, una tarjeta cerrable "Primeros pasos", la configuración inicial en 5 pasos con validación y resumen editable, errores de autenticación traducidos al español sin revelar cuál de los dos datos es el incorrecto, recuperación de contraseña por email, y avisos aclarados (el umbral de inactividad solo muestra un recordatorio: no borra nada). Las URLs anteriores siguen funcionando con redirecciones.

Una decisión con alternativa fue el largo mínimo de la contraseña. Se subió a 8 en registro, recuperación y Mi cuenta, pero el inicio de sesión no lo exige, para que las cuentas anteriores sigan entrando. El medidor de fortaleza orienta, pero no bloquea.

5.12 Cabeceras de seguridad y política de contenido

Producción enviaba solo la cabecera de transporte seguro que agrega Vercel; faltaban las que protegen al navegador. Se configuraron en vercel.json para todas las rutas: una política de contenido (CSP) cerrada por defecto, protección contra el clickjacking (X-Frame-Options y frame-ancestors en «none»), nosniff, una política de referer, una política de permisos que apaga la cámara, el micrófono y la geolocalización, y el aislamiento de la ventana (COOP).

Decisión con alternativa: los scripts solo se permiten desde el propio sitio, sin «unsafe-inline» ni «unsafe-eval», que es lo que realmente frena un XSS. En cambio los estilos sí permiten «unsafe-inline»: la alternativa, un nonce distinto por respuesta, no es posible con un sitio estático servido por Vercel, y React escribe estilos en línea (por ejemplo en las barras de progreso y en los gráficos). Es un riesgo menor, porque un estilo no ejecuta código. Las conexiones salen solo hacia Supabase (https y wss) y las dos fuentes de ofertas, y Google Fonts es el único recurso externo permitido.

La política se verificó contra la aplicación real y no solo escrita: se hizo un build de producción, se lo sirvió con exactamente esas cabeceras y se recorrieron todas las pantallas del candidato y del administrador, incluidos los gráficos y las fuentes, sin una sola violación. Un test unitario lee vercel.json y falla si alguien afloja la política.

5.13 Plantillas de mensaje con campos por completar

Para que escribirle a un candidato sea simple, el administrador puede partir de cinco plantillas (por ejemplo, «Solicitud recibida» y «Falta un dato»). Saludan por el primer nombre y evitan el género de la persona, porque el nombre no lo indica. Lo que va entre corchetes es un dato por completar: queda seleccionado para escribir encima, y el envío se bloquea mientras haya corchetes sin reemplazar, para que nunca salga un mensaje con un «[indicá cuál]» a medias. Aplicar una plantilla sobre un texto ya escrito se puede deshacer.

La recuperación de contraseña también se rediseñó en tres pasos visibles (pedir el enlace, revisar el correo y elegir la contraseña), con una pantalla «Revisá tu correo» que explica qué hacer si el mail no llega, un reenvío con cuenta regresiva (el servicio permite un envío por minuto), una lista de requisitos que se tilda en vivo y un mail propio en español (docs/plantillas-email), cuyo contenido cuida un test.

6. Seguridad y privacidad

Esta sección reúne cómo se protegen los datos de cada persona y qué puede ver cada rol. El diseño se detalla en las Secciones 5.7 a 5.10; acá se resume el resultado y cómo se verificó.

Modelo de acceso

- Candidato: ve y modifica únicamente lo suyo. Todas las tablas de dominio tienen RLS por user_id, igual que antes de la ampliación.
- Administrador: no tiene política sobre ninguna tabla personal. Solo lee la vista candidatos y puede enviar mensajes de tipo admin firmados con su propio id.
- Sin sesión: no accede a nada de lo nuevo (roles, mensajes ni candidatos).
- Permisos mínimos por objeto: roles_usuario solo SELECT; candidatos solo SELECT; mensajes SELECT, INSERT y DELETE, y UPDATE únicamente de la columna leido; es_administrador() solo ejecutable por usuarios con sesión; la función del trigger de bienvenida no es ejecutable por la API.

Qué datos ve el administrador

- Ve: el nombre y el perfil de búsqueda de cada candidato que completó su perfil (rol buscado, tecnologías, modalidad, ubicación, nivel de experiencia) y la fecha de registro. Los candidatos que omitieron el Onboarding no se listan.
- No ve: ofertas, postulaciones, interacciones, recordatorios, criterios de scoring, historial de estados, el email, el umbral de recordatorios ni los mensajes de otras personas.

Aviso al usuario

Cada persona ve este aviso antes de dar sus datos y cada vez que los revisa: "Tu nombre y tu perfil de búsqueda (rol, tecnologías, modalidad, ubicación y nivel) los puede ver el equipo de Laburin. Tus ofertas y postulaciones son solo tuyas." Se muestra en el primer paso de la configuración inicial (junto al nombre), en el resumen final y en Mi cuenta.

Pruebas de seguridad

Una suite adversarial de 13 pruebas corre contra la base real, sin navegador y sin service role key: usa la clave pública con sesiones reales de dos candidatos creados en cada corrida y de la cuenta de administrador de pruebas, es decir, con las mismas reglas que tiene un usuario de la aplicación. Cada ataque se considera exitoso solo si es rechazado (error de permisos o de política) o queda sin efecto (0 filas), y en los casos de escritura se verifica además que el dato ajeno siga intacto. Las 13 pasaron. Cubren:

- Candidato contra candidato: no lee perfil, mensajes, ofertas, postulaciones, interacciones, recordatorios, criterios ni historial ajenos; no modifica ni borra datos ajenos; no inserta filas a nombre de otro.

- Escalada de rol: un candidato no puede insertar, hacer upsert, actualizar ni borrar en roles_usuario, y es_administrador() le devuelve falso. La vista candidatos le devuelve 0 filas y rechaza cualquier escritura.
- Mensajes: de un mensaje solo se puede cambiar el estado leído; no se puede editar el texto, el remitente ni el destinatario, ni enviar a nombre del sistema o de otra persona.
- Administrador: lee solo las columnas acordadas de candidatos (pedir otras da error de columna inexistente) y 0 filas de todo lo privado, incluida una oferta y una nota sembradas a propósito; no puede insertar, actualizar ni borrar en candidatos, perfil_usuario ni roles_usuario, ni cambiar su propio rol; envía como él mismo y no puede suplantar remitente o tipo, ni leer, editar, marcar o borrar lo ajeno.
- Sin sesión: todo lo nuevo da error de permisos, y perfil_usuario devuelve 0 filas.
- Bienvenida: se envía una sola vez al completar el perfil con un rol, no se duplica y no se envía a quien omite el Onboarding.

Revisión de seguridad de la aplicación

Además de la base de datos, se revisó la aplicación web con la metodología de auditoría de cabeceras y de pruebas de XSS, y se corrigió lo encontrado:

- Cabeceras HTTP: producción enviaba solo HSTS. Se agregaron la política de contenido y el resto (Sección 5.12), verificadas sin violaciones y protegidas por un test.
- Inyección de código (XSS): el código fuente no usa innerHTML ni dangerouslySetInnerHTML, y no hay enlaces externos que abran pestañas sin protección. Una prueba con navegador real carga código malicioso en el nombre, el rol, la ubicación y las tecnologías de un candidato (lo que después ve el administrador), en un mensaje y en ofertas importadas, y comprueba que todo se ve como texto y nada se ejecuta. Se comprobó también que la prueba detecta el fallo: al introducirlo a propósito, se pone en rojo.
- Dependencias: se corrigió una vulnerabilidad alta (source-map-js, denegación de servicio). Para lo que se despliega, la auditoría de npm informa 0 vulnerabilidades; quedan 3 moderadas en una herramienta de desarrollo (la que genera los PDF), que no forma parte de la aplicación.
- Secretos: no hay claves en el historial de git (se buscaron claves JWT, service role y archivos .env versionados); la aplicación solo usa la clave pública anon.
- Accesibilidad y diseño responsive: una auditoría automática (axe-core, WCAG A y AA) de todas las pantallas y los modales, en tema claro y oscuro, encontró un único problema de contraste, ya corregido; y una prueba verifica que ninguna pantalla se salga del ancho de un celular de 320 px. Ambas quedan como pruebas permanentes.
- Pendiente del lado del panel de Supabase (no se puede hacer desde el código): largo mínimo de contraseña de 8 en el servidor, protección contra contraseñas filtradas si el plan la incluye y el correo propio. Están explicados paso a paso en docs/guia-supabase.md.

Límites

- El rol administrador se asigna por SQL y depende de quien tenga acceso al panel de la base; no hay una pantalla para gestionarlo.

- El administrador ve el nombre que cada persona escribió; no se verifica que sea su nombre real.
- Los datos de pruebas usan cuentas propias y se borran al terminar. Los usuarios de Auth de prueba quedan, porque borrarlos requeriría una service role key, que el proyecto no usa.

7. Testing

Metodología

Las funciones puras del dominio (motor de scoring, criterios, recordatorios, métricas, utilidades) se desarrollaron bajo TDD con Vitest: test en rojo antes de la implementación, luego verde. Todo lo que es integración UI–backend se verificó, a falta de navegador disponible durante gran parte del desarrollo, mediante una combinación de lectura de código y verificación directa contra una instancia real de Supabase — nunca contra mocks. En el pase final se sumó una suite de Playwright con un navegador real (Chromium), separada de Vitest: npm run test corre solo los tests unitarios y npm run test:e2e corre la suite de punta a punta.

Pase de QA

Auditoría sistemática de toda la app por lectura de código, clasificada en bloqueante / importante / menor. No se encontraron bloqueantes. Se corrigieron 3 hallazgos importantes — el campo de peso "pegado" en 0 al editar un criterio existente, ausencia de límites mínimo/máximo en los campos de peso (un peso negativo restaba puntos de forma silenciosa), y el botón "Cancelar" del modal de oferta que no bloqueaba un guardado ya en curso — y 2 menores: ausencia de maxLength en todos los campos de texto, y falta de feedback visual durante la verificación inicial de sesión. Dos hallazgos menores quedaron en ese momento fuera de alcance a pedido explícito — duplicados casi-idénticos de tags de stack por mayúsculas o espacios, y el límite de 1000 filas por defecto de Supabase sin paginar — y se resolvieron en el pase final (Secciones 5.6 y 7, Pase con Playwright).

Pase E2E

Seis flujos de punta a punta (alta de usuario, ciclo de vida completo de una oferta, importación con scoring y filtros, configuración afectando el scoring en vivo, recordatorios por inactividad, sesión y navegación) verificados con dos fuentes de evidencia combinadas: un script temporal contra la instancia real de Supabase de producción (con autorización explícita), que ejecutó 33 verificaciones concretas contra las mismas tablas, triggers y RLS que usa la aplicación — las 33 en verde — y trazado de código para la parte puramente de React (qué se re-renderiza y qué se vuelve a pedir al navegar entre pantallas). No se encontraron hallazgos nuevos. El reporte documenta explícitamente lo que esta metodología no puede cubrir — la sensación real del drag-and-drop, el timing exacto de un click, el comportamiento de una sesión abierta muchas horas — como zonas ciegas honestas, no como bugs descartados. El pase con Playwright, descrito a continuación, cerró varias de ellas.

Pase con Playwright

Una suite de Playwright con Chromium real, contra la aplicación levantada con Vite y la base real de Supabase, en dos proyectos: desktop (1280x800) y mobile (375x812, con touch). Corre con un solo worker porque comparte base y una cuenta por proyecto. Las cuentas de prueba se crean por el propio flujo de registro de la aplicación, las credenciales viven en un archivo de entorno que no se versiona, y la siembra y la limpieza de datos se hacen con el cliente de Supabase autenticado como el

usuario de prueba — es decir, respetando RLS, sin ninguna service role key. Al terminar, un teardown borra todos los datos de las cuentas de prueba.

Cubre: alta de usuario completa (registro, configuración inicial de 5 pasos con doble tap en "Continuar" en mobile, Mis postulaciones vacío, criterios reflejados en Mi perfil de búsqueda); el ciclo de vida de una oferta (crear, ver el score, postular, moverla por las 6 columnas con drag-and-drop real en desktop y con el selector "Mover a..." en mobile, Ficha con interacciones, persistencia tras recargar); importación, filtros combinados y borrado masivo (también con la barra de selección en mobile); el efecto de un cambio de peso y de un criterio nuevo sobre el score; el umbral de inactividad (persistencia, validación y momento en que aparece el recordatorio); más de 1000 ofertas (paginación completa, filtros más allá de la fila 1000, contador, selección de las N que coinciden, deduplicación del import, Analytics y recálculo de scores); Mi cuenta (nombre, tema, contraseña); y sesión (cerrar, reingresar, rutas protegidas sin sesión). Las cifras de la corrida final, que incluye los pases posteriores, están en el Pase de claridad y roles.

Hallazgos reales que encontró y que se corrigieron, cada uno con su test: (1) una carrera en el Tablero — una lectura inicial tardía pisaba el primer movimiento y devolvía la tarjeta a su columna aunque la base ya la había guardado; se corrigió descartando la respuesta de una corrida cancelada y se cubrió con un test que reproduce la respuesta vieja de forma determinista (falla sin el arreglo, pasa con él); (2) controles sin nombre accesible — pesos, botones de borrar y agregar de Configuración, filtros de Ofertas, el estado de la Ficha, los botones de cerrar — y modales sin role="dialog"; (3) el resumen de la importación decía "1 ofertas nuevas importadas". Además, una de las fallas intermitentes resultó ser un error de sincronización del propio test y se corrigió ahí, sin tocar la aplicación.

Zonas ciegas del pase anterior que se cerraron: la sensación real del drag-and-drop en desktop (incluido el recorrido por las 6 columnas) y el timing de los clicks, ahora con un navegador real; el doble tap en el Onboarding; el selector "Mover a..." y la barra de selección en una pantalla de 375 px. Zonas ciegas que siguen abiertas: el teclado virtual real (solo se simula su efecto reduciendo el viewport; no se puede automatizar con fiabilidad); el comportamiento en un celular real y en otros motores (solo se probó Chromium, no WebKit/Safari ni hardware real); el autoscroll del navegador al arrastrar una tarjeta cerca del borde (el test desplaza el tablero antes de arrastrar); y la disponibilidad de Remotive y Arbeitnow (el import usa respuestas simuladas, con un único test de humo contra los servicios reales).

Pase de claridad y roles

Con la devolución de la docente y la ampliación del 08/10/2026 se sumaron especificaciones de Playwright que cubren lo nuevo: navegación y redirecciones de las URLs anteriores; acceso y recuperación de contraseña (validaciones en español, errores traducidos, mensaje neutro, enlace vencido y nueva contraseña, con el envío de mails interceptado para no depender de ningún correo real ni de sus límites); filtros de Ofertas (ubicación con sugerencias, puntaje mínimo según el perfil, sin resultados) y aviso de perfil vacío; configuración inicial de 5 pasos con validaciones, texto sin agregar y duplicados; un recorrido completo de una persona que no conoce la aplicación, desde un registro con errores a propósito hasta volver a entrar; el flujo entre candidato y administrador (bienvenida, listado, envío de un mensaje, no leído, marcado como leído, y el acceso por rol, incluido el intento de entrar a Candidatos por URL); y la suite adversarial de seguridad de la Sección 6.

Cifras de la corrida final completa (desktop y mobile, contra la base real): 277 tests unitarios de Vitest en 26 archivos y 112 tests de Playwright, ninguno salteado. De los 112, 2 son sesiones de preparación, 61 corren en desktop (13 de ellos son las pruebas de seguridad contra la base, que hablan directo con la API y por eso corren una sola vez) y 49 corren en mobile. En la corrida completa pasaron 111 y uno falló una vez por una interrupción de red; repetido solo, pasó las 3 veces. El test del teclado virtual solo aplica a mobile y se excluye del proyecto desktop por configuración, en lugar de figurar como salteado en cada corrida.

Hallazgos de este pase que se corrigieron: los textos de interfaz que seguían en inglés o con jerga ("pipeline", "Timeline", "Stack tecnológico", "Score", "pts"), los mensajes de error de autenticación en inglés y un ayudante de las pruebas que intentaba abrir el menú móvil ya abierto. Además, las credenciales de la cuenta de administrador de pruebas se ubicaron por error en un archivo de ejemplo versionado; se detectó antes de commitear, se movieron al archivo de entorno que no se versiona y nunca llegaron al repositorio.

Bugs de producción

Aparte de los dos pases anteriores, el uso real en dispositivos móviles encontró y corrigió 4 bugs puntuales: el teclado virtual tapando el campo enfocado, un doble tap en el Onboarding que dejaba dos pantallas superpuestas, la lista de Ofertas sin refrescar tras importar, y un problema de layout responsivo en la barra de selección múltiple de Ofertas.

8. Trabajo fuera del alcance original

Cosas construidas durante el desarrollo que no estaban en la especificación original, o que la contradicen explícitamente:

- Multi-usuario funcional completo: la SRS (§7.3, RNF-02) preveía dejar el modelo de datos preparado para multi-usuario, pero explícitamente no construir pantallas de registro público ni onboarding genérico durante esta PPS — el sistema se evaluaría con el perfil único del autor. En la práctica se construyó el flujo completo de registro, login, recuperación de contraseña y configuración inicial de 5 pasos, y el proyecto se evalúa como multi-usuario real.
- Pase de usabilidad y mobile completo: sidebar responsivo con drawer, el selector como fallback del Kanban en mobile (Sección 5.1), ajuste de touch targets a 44px, y visibilidad de botones hover-only en pantallas táctiles. No estaba contemplado en los requerimientos funcionales originales.
- Identidad visual: medidor de fortaleza de contraseña, mostrar/ocultar contraseña, fondo ambient con formas difuminadas (más marcado en Login/Registro, atenuado en el resto de la app) y tema claro/oscuro. Trabajo de pulido visual no pedido por la SRS.
- postulacion_historial_estados: tabla y trigger adicionales para registrar automáticamente cada cambio de estado de una postulación, no pedidos explícitamente en la SRS — se agregaron para sostener con datos reales el Panel de Métricas (tiempo promedio por etapa).
- Rol de administrador, pantalla Candidatos y mensajes con bienvenida automática: ampliación acordada con la docente el 08/10/2026 (Secciones 5.7 a 5.10 y 6).
- Pase de claridad pedido por la docente: nombres de secciones, ayuda en cada pantalla, estados vacíos, "Primeros pasos", validación y errores en español, recuperación de contraseña (Sección 5.11).
- Cuatro pases de verificación completos (QA, E2E contra la base real, una suite de Playwright con navegador real en desktop y mobile, y pruebas adversariales de seguridad contra la base real), además de los tests unitarios — un nivel de rigor de verificación no exigido explícitamente por la especificación funcional.
- Paginación en servidor del listado de Ofertas y lecturas por lotes en todo el sistema, para que la aplicación siga correcta pasado el límite de 1000 filas por lectura de Supabase (Sección 5.6).

9. Limitaciones conocidas y trabajo futuro

- Recálculo de scores lento con volúmenes grandes: cambiar un peso con más de 1000 ofertas actualiza cientos de filas, de a diez en paralelo, y en las pruebas con 1100 ofertas tardó entre 25 segundos y 1,2 minutos según la corrida (más de un minuto en la medición más lenta). Es correcto pero no instantáneo. Mejora posible: una función SQL que recalcule todos los puntajes en una sola sentencia, sin una escritura por fila.
- Importar ofertas inserta una por una (cada alta lee la sesión y los criterios): con muchas ofertas nuevas es lento. Un insert por lotes lo mejoraría.
- Sin pruebas en un celular real ni en otros motores, y sin automatizar el teclado virtual real: la suite de Playwright usa Chromium emulado y solo simula el efecto del teclado (Sección 7).
- Solo 2 de las 7 fuentes originales, sin automatización periódica: la búsqueda de ofertas externas depende de que el usuario toque "Importar" — no hay job de fondo (Sección 5.2 y 5.3).
- Sin notificación por email: los recordatorios solo viven dentro de la aplicación (badge en Tablero y Ficha), no hay el mail de resumen diario que tenía el antecedente.
- El motor de scoring no portó las reglas fijas heredadas del script anterior (recencia, seniority, exclusiones por keyword, idioma, salario): el puntaje depende exclusivamente de los criterios que el usuario configure. Si el usuario no configura ningún criterio, no hay ninguna señal de calidad de base.
- La recuperación de contraseña depende del correo configurado en Supabase: con el servicio por defecto solo reciben el mail los miembros de la organización del proyecto y el tope es de 2 mensajes por hora, además de una espera de 60 segundos por usuario. Para usarla con personas reales hace falta un servicio de correo propio (SMTP).
- El rol de administrador se asigna a mano por SQL: no hay pantalla para gestionarlo ni para quitarlo.
- Los mensajes son de una sola vía (del equipo al candidato) y no hay notificación por email ni actualización en vivo: el contador de no leídos se actualiza al navegar, al volver a la pestaña y al operar en Mensajes.
- El candidato puede borrar los mensajes que recibe y el administrador pierde ese mensaje de su historial de enviados.
- Quienes omiten la configuración inicial no tienen perfil de búsqueda: no se listan en Candidatos y no reciben la bienvenida. Las cuentas anteriores a la ampliación no tienen nombre hasta que lo completen en Mi cuenta.
- El indicador de fortaleza de contraseña orienta pero no bloquea: con 8 caracteres o más se puede elegir una contraseña que el medidor marca como débil.
- La política de contenido permite estilos en línea («unsafe-inline» solo para estilos, nunca para scripts): React y los gráficos los necesitan y un sitio estático no puede usar un nonce por respuesta (Sección 5.12).
- La auditoría de accesibilidad es automática (axe-core): detecta los errores más comunes, pero no reemplaza una revisión manual con lector de pantalla ni con usuarios reales.
- Sin categorización de rol (QA / Data Entry / Admin / Data Analyst, u otra taxonomía): no hay forma de filtrar o tagear ofertas por categoría de puesto más allá de los criterios de texto libre que arme

el propio usuario.

10. Conclusiones

Esta sección compara, punto por punto, lo entregado contra la especificación de requerimientos (SRS) original. El objetivo general del proyecto — una plataforma con motor de scoring configurable y seguimiento activo de postulaciones, reemplazando el script y la planilla del antecedente — se cumplió. Lo que sí varió respecto del documento inicial fue el reparto interno de esfuerzo: se priorizó profundidad en scoring, tablero, ficha de postulación y calidad vía testing, por sobre completitud de automatización (fuentes externas, ejecución periódica, notificaciones por email).

Requerimientos funcionales

Req.	Estado	Nota
RF-01	**Parcial**	Solo 2 de las 7 fuentes (Remotive, Arbeitnow); importación manual, no automatizada ni periódica.
RF-02	**Cumplido**	Aislamiento por fuente (try/catch), aplicado a las 2 fuentes implementadas.
RF-03	**No implementado**	Sin categorización QA/Data Entry/Admin/Data Analyst — descartado a propósito en favor del scoring configurable como discriminador principal.
RF-04	**Parcial**	Deduplicación por empresa+rol+fuente sí implementada; vencimiento a 72hs no implementado.
RF-05	**Parcial**	El score surge solo de los 3 criterios configurables; las reglas fijas heredadas (recencia, seniority, exclusiones, idioma, salario) no se portaron al motor.
RF-06	**Cumplido**	Alta manual e importación conviven como vías complementarias.
RF-07	**Cumplido**	Listado filtrable por fuente, ubicación y puntaje mínimo (el control llega hasta el máximo posible con el perfil de cada usuario).
RF-08	**Cumplido**	"Postularme" crea la Postulación en estado por_aplicar.
RF-09	**Cumplido**	Tablero de 6 columnas de estado.
RF-10	**Cumplido (ampliado)**	Drag nativo en desktop, más un selector como fallback en mobile no pedido originalmente (Sección 5.1).
RF-11	**Cumplido**	Ficha con historial de interacciones por fecha y tipo.

Req.	Estado	Nota
RF-12	**Cumplido**	Los 3 criterios (stack, modalidad, ubicación) son editables sin tocar código.
RF-13	**Cumplido, con matiz**	Las reglas fijas no se exponen en la UI, pero tampoco llegaron a implementarse en el motor — no es que estén ocultas, es que no existen (ver RF-05).
RF-14	**Cumplido**	El umbral de inactividad se configura por usuario (de 1 a 90 días, 7 por defecto), se guarda en la base y es el que usa el motor de recordatorios. El cambio aplica hacia adelante (Sección 5.5).
RF-15	**Cumplido**	Se generan recordatorios, con el umbral configurable de cada usuario, cuando una postulación queda sin novedades.
RF-16	**No implementado**	Sin notificación por email.
RF-17	**Cumplido**	Panel de Métricas con indicadores operativos reales.

Requerimientos no funcionales

Req.	Estado	Nota
RNF-01	**Cumplido**	Aislamiento por fuente, aplicado a las 2 fuentes reales implementadas.
RNF-02	**Ampliado**	Se construyó multi-usuario funcional completo (registro, login, onboarding), más allá de lo que la propia SRS preveía para esta PPS (§7.3 decía explícitamente que no se construiría).
RNF-03	**Cumplido**	Autenticación vía Supabase Auth.
RNF-04	**Cumplido**	Persistencia relacional en Postgres, con relación real Oferta↔Postulación (UNIQUE).
RNF-05	**No cumplido**	Sin cron ni Edge Function de búsqueda; la importación es manual. El GitHub Action existente es solo un keepalive de Supabase (Sección 5.2), no cumple la función de búsqueda periódica.

Ampliación acordada con la docente (08/10/2026)

Req.	Estado	Nota
AMP-01	**Cumplido**	Rol de administrador asignado por SQL; no se obtiene desde la interfaz ni se puede escalar desde la API (Sección 5.7).
AMP-02	**Cumplido**	Pantalla Candidatos con búsqueda y detalle, limitada al perfil de búsqueda; no se listan perfiles sin rol buscado (Sección 5.8).
AMP-03	**Cumplido**	Mensajes del administrador al candidato, con contador de no leídos, marcado como leído y borrado.
AMP-04	**Cumplido**	Mensaje de bienvenida automático al completar el perfil (trigger, Sección 5.9).
AMP-05	**Cumplido**	Aviso de privacidad en la configuración inicial y en Mi cuenta (Sección 6).
AMP-06	**Cumplido, con matiz**	Recuperación de contraseña por email con mensaje neutro; con el correo por defecto de Supabase tiene límites de envío (Sección 9).
AMP-07	**Cumplido**	Claridad de la interfaz: nombres, ayuda, estados vacíos, "Primeros pasos", validación y errores en español (Sección 5.11).
AMP-08	**Cumplido**	Plantillas de mensaje para el administrador, con campos por completar y deshacer (Sección 5.13).

En síntesis: de los 17 RF originales, 12 se cumplieron (10 tal como fueron especificados, 1 ampliado — RF-10 — y 1 cumplido con matices — RF-13), 3 se cumplieron parcialmente (RF-01, RF-04, RF-05) y 2 no se implementaron (RF-03, RF-16). De los 5 RNF, 3 se cumplieron, 1 se amplió deliberadamente por sobre lo previsto (RNF-02) y 1 no se cumplió (RNF-05). A eso se suman las 8 ampliaciones acordadas con la docente el 08/10/2026: 7 cumplidas y 1 con matices (la recuperación de contraseña depende del correo configurado). Ninguno de los recortes fue accidental: cada uno queda documentado con su alternativa descartada en la Sección 5 o como limitación conocida en la Sección 9, en vez de quedar implícito.